

Qt + Elastic/Kibana: Production Architecture

Matrix-inspired E2E chat and observability are separate paths. The Kibana-in-Qt prototype evolves into the metrics path, not the chat transport.

1. Chat path

E2E CHAT PATH

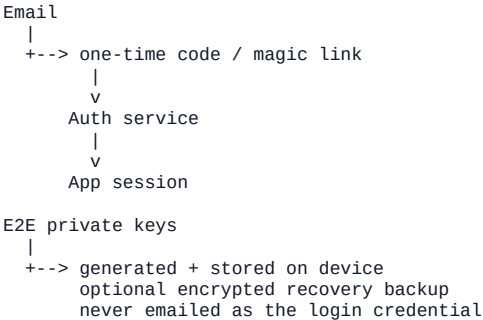


Reverse direction works the same way.

Elasticsearch and Kibana are not in the message path. The backend handles ciphertext; clients handle plaintext locally.

3. Authentication

ACCOUNT / E2E KEY SEPARATION



2. Observability path

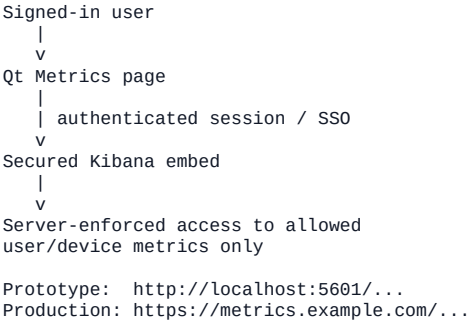
OBSERVABILITY PATH



Central Elasticsearch handles metric history, search, aggregation, and retention. Kibana can remain embedded in Qt for the metrics page.

4. User-facing metrics

USER METRICS IN QT



Production checks

Area	Production requirement
Agent management	For many endpoints, use Fleet/Fleet Server for centralized agent policy and lifecycle management; standalone agents are still valid for smaller deployments.
Agent -> Elastic	Use TLS and least-privilege API keys/service credentials. Do not ship shared admin credentials in the desktop app.
Kibana embed	Use HTTPS plus authenticated Kibana access/SSO and read-only privileges. Configure embedded-session cookies correctly.
Data isolation	Enforce user/device access server-side. A dashboard filter by itself is not an authorization boundary.
Telemetry privacy	Exclude plaintext messages, E2E private keys, and unnecessary sensitive chat metadata from telemetry.
Scale / storage	Use retention, aggregation/downsampling, and metrics-oriented Elasticsearch data streams as volume grows.
Refresh	10-second refresh is reasonable for host metrics. Add a separate realtime stream only if lower latency becomes a real requirement.